

Zajęcia P2. Analizator składniowy dla uproszczonego języka Ada

1 Cel ćwiczeń

Celem ćwiczeń jest stworzenie prostego analizatora składniowego dla bardzo uproszczonej wersji języka Ada. Zadaniem tworzonego analizatora jest:

- rozpoznawanie konstrukcji składniowych języka Ada
- wykrywanie błędów składniowych

2 Czynności wstępne

Po włączeniu komputera należy wybrać system operacyjny Linux i zalogować się jako użytkownik `student`. Należy otworzyć okno konsoli (np. nacisnąć `Alt-F2` i napisać `xterm`), utworzyć własny podkatalog za pomocą polecenia `mkdir nazwisko_uzytkownika`, i podkatalog dla bieżącego ćwiczenia. Ze strony przedmiotu na platformie Moodle należy pobrać następujące pliki:

- `Makefile` — potrzebny do kompilacji za pomocą polecenia `make`
- `ada.y` — szcątkowy analizator składniowy z komentarzami i zdefiniowaną funkcją `found`
- `listascii.adb` — program testowy poprawny w danej gramatyce
- `test.adb` — program testowy poprawny w danej gramatyce

Po zakończeniu pracy wskazane jest usunięcie utworzonego katalogu wraz z zawartością.

3 Zadania do wykonania

Do wykonania zadania potrzebny jest analizator leksykalny przygotowany w poprzednim ćwiczeniu. W przypadku braków w analizatorze leksykalnym należy je uzupełnić. Należy uzupełnić dostarczony szkielet analizatora składniowego i pokazać, że działa poprawnie testując go na dostarczonych programach testowych. Analizator powinien wypisywać informacje o rozpoznanych konstrukcjach składniowych. Do wypisywania tych informacji służy zawarta w dostarczonym szkielecie analizatora funkcja `found()`. Funkcja ma dwa parametry: nazwa rozpoznanej konstrukcji (należy tu wpisać nazwę zmiennej gramatyki) oraz argument, który ma znaczenie (jest różny od pustego napisu) dla niektórych konstrukcji, np. może być nazwą funkcji. Należy dążyć do uzyskania takiego samego wyjścia programu jak w punkcie 6.

Dostarczony kod należy uzupełnić o następujące elementy:

- A. klauzula `with` (`WITH_CLAUSE`)
- B. klauzula `use` (`USE_CLAUSE`)
- C. specyfikacja kontekstu (`CONTEXT_SPEC`)
- D. nagłówek procedury (`PROC_HEADER`)
- E. deklaracja procedury (`PROC_DECL`)
- F. deklaracja pakietu (`PACKAGE_DECL`)
- G. deklaracja zmiennej (`VAR_DECL`)
- H. wywołanie procedury (`PROCEDURE_CALL`)
- I. pętla `for` (`FOR_LOOP`)
- J. przypisanie (`ASSIGNMENT`)
- K. blok (`BLOCK`)
- L. definicja procedury (`PROC_DEFINITION`)
- M. ciało pakietu (`PACKAGE_BODY`)
- N. deklaracja stałej (`CONST_DECL`)
- O. instrukcja warunkowa (`IF_STATEMENT`)

4 Ocena

Za każdy można dostać 1 punkt, czyli 15 punktów na zajęciach. Elementy od A do J muszą być wykonane na zajęciach. Pozostałe elementy można oddać w domu po 0,5 punktu (razem 2,5 punktu). Punkty będą przyznawane dopiero po rozmowie z prowadzącym.

5 Dane testowe — plik test.adb

```
1  — Ten program drukuje znaki i ich kody ASCII
2  — Kompilacja: gnatmake test
3
4  with Ada.Text_IO; use Ada.Text_IO;
5  with Ada.Integer_Text_IO;
6
7  procedure Test is
8
9      package MyTest is
10         procedure ListAscii(LowerBound: in Integer; UpperBound: in Integer);
11     end MyTest;
12
13     package body MyTest is
14         procedure ListAscii(LowerBound: in Integer; UpperBound: in Integer) is
15             Uc : Character := ' ';           — zmienna
16             Fl : Float;
17             T : array(1..10) of Float;
18             I : Integer;
19             D : record
20                 Dzień, Miesiąc : Integer;
21                 Rok : Integer;
22             end record;
23         begin
24             New_Line;
25             Put_Line("Kody ASCII:");
26             for I in Integer range LowerBound .. UpperBound loop
27                 Ada.Integer_Text_IO.Put(I);   — procedura z Ada.Integer_Text_IO
28                 Put(' ');   — to jest procedura z Ada.Text_IO
29                 Put(Uc);
30                 Uc := Character'Succ(Uc);     — kolejny znak
31                 New_Line;
32             end loop;
33             — tylko test
34             Fl := 1.1 * 0.1 + 1.0e-2; — po kropce i przed zawsze cyfra
35             T(0) := 0;
36             for I in 1 .. 10 loop
37                 T(I) := T(I) * I * I;
38             end loop;
39             D.Dzień := 1;
40             D.Miesiąc := D.Dzień * 10;
41             D.Rok := 2018;
42         end ListAscii;
43
44     end MyTest;
45
46     use MyTest;
47
48     FromASCII : constant Integer := 32;
49     ToASCII : constant Integer := 127;
50
51
52     begin
53         if FromAscii <= ToAscii then
54             ListAscii(FromAscii, ToAscii);
55         else
56             Put_Line("Figa");
```

```

57     New_Line;
58   end if;
59 end Test;

```

6 Wyjście analizatora składniowego dla test.adb

```

1  Imie i Nazwisko
2  yytext          Typ tokena      Wartosc tokena znakowo
3
4  with            KW.WITH
5  Ada             IDENT            Ada
6  .
7  Text_IO         IDENT            Text_IO
8  ;
9  ===== FOUND: WITH_CLAUSE =====
10 use             KW.USE
11 Ada             IDENT            Ada
12 .
13 Text_IO         IDENT            Text_IO
14 ;
15 ===== FOUND: USE_CLAUSE =====
16 ===== FOUND: CONTEXT_SPEC =====
17 with            KW.WITH
18 Ada             IDENT            Ada
19 .
20 Integer_Text_IO IDENT            Integer_Text_IO
21 ;
22 ===== FOUND: WITH_CLAUSE =====
23 procedure       KW.PROCEDURE
24 ===== FOUND: CONTEXT_SPEC =====
25 Test            IDENT            Test
26 is              KW.IS
27 ===== FOUND: PROC_HEADER 'Test' =====
28 package         KW.PACKAGE
29 MyTest          IDENT            MyTest
30 is              KW.IS
31 procedure       KW.PROCEDURE
32 ListAscii       IDENT            ListAscii
33 (
34 LowerBound      IDENT            LowerBound
35 :
36 in              KW.IN
37 Integer         IDENT            Integer
38 ;
39 UpperBound      IDENT            UpperBound
40 :
41 in              KW.IN
42 Integer         IDENT            Integer
43 )
44 ===== FOUND: PROC_HEADER 'ListAscii' =====
45 ;
46 ===== FOUND: PROC_DECL 'ListAscii' =====
47 end             KW.END
48 MyTest          IDENT            MyTest
49 ;
50 ===== FOUND: PACKAGE_DECL 'MyTest' =====
51 package         KW.PACKAGE
52 body            KW.BODY
53 MyTest          IDENT            MyTest
54 is              KW.IS
55 procedure       KW.PROCEDURE
56 ListAscii       IDENT            ListAscii
57 (
58 LowerBound      IDENT            LowerBound

```

```

59 :                               :
60 in                             KW.IN
61 Integer                         IDENT          Integer
62 ;                               ;
63 UpperBound                     IDENT          UpperBound
64 :                               :
65 in                             KW.IN
66 Integer                         IDENT          Integer
67 )                               )
68 ===== FOUND: PROC.HEADER 'ListAscii' =====
69 is                             KW.IS
70 Uc                             IDENT          Uc
71 :                               :
72 Character                       IDENT          Character
73 :=                             ASSIGN
74 ' '                             CHARACTER.CONST ' '
75 ;                               ;
76 ===== FOUND: VAR.DECL 'Uc' =====
77 Fl                             IDENT          Fl
78 :                               :
79 Float                          IDENT          Float
80 ;                               ;
81 ===== FOUND: VAR.DECL 'Fl' =====
82 T                              IDENT          T
83 :                               :
84 array                          KW.ARRAY
85 (                              (
86 1                              INTEGER.CONST 1
87 ..                             RANGE
88 10                             INTEGER.CONST 10
89 )                              )
90 of                             KW.OF
91 Float                          IDENT          Float
92 ;                               ;
93 ===== FOUND: VAR.DECL 'T' =====
94 I                              IDENT          I
95 :                               :
96 Integer                         IDENT          Integer
97 ;                               ;
98 ===== FOUND: VAR.DECL 'I' =====
99 D                              IDENT          D
100 :                              :
101 record                         KW.RECORD
102 Dzien                          IDENT          Dzien
103 ,                              ,
104 Miesiac                       IDENT          Miesiac
105 :                              :
106 Integer                         IDENT          Integer
107 ;                              ;
108 Rok                           IDENT          Rok
109 :                              :
110 Integer                         IDENT          Integer
111 ;                              ;
112 end                            KW.END
113 record                         KW.RECORD
114 ;                              ;
115 ===== FOUND: VAR.DECL 'D' =====
116 begin                          KW.BEGIN
117 New_Line                       IDENT          New_Line
118 ;                              ;
119 ===== FOUND: PROCEDURE.CALL 'New_Line' =====
120 Put_Line                       IDENT          Put_Line
121 (                              (
122 "Kody ASCII:"                 STRING.CONST  "Kody ASCII:"
123 )                              )
124 ;                              ;

```

```

125 ===== FOUND: PROCEDURE_CALL 'Put_Line'=====
126 for                KWFOR
127 I                  IDENT            I
128 in                 KW.IN
129 Integer            IDENT            Integer
130 range              KWRANGE
131 LowerBound         IDENT            LowerBound
132 ..                 RANGE
133 UpperBound         IDENT            UpperBound
134 loop               KWLOOP
135 Ada                IDENT            Ada
136 .
137 Integer_Text_IO    IDENT            Integer_Text_IO
138 .
139 Put                IDENT            Put
140 (                  (
141 I                  IDENT            I
142 )                  )
143 ;
144 ===== FOUND: PROCEDURE_CALL 'Ada.Integer_Text_IO.Put'=====
145 Put                IDENT            Put
146 (                  (
147 ' ,                CHARACTER_CONST ' '
148 )                  )
149 ;
150 ===== FOUND: PROCEDURE_CALL 'Put'=====
151 Put                IDENT            Put
152 (                  (
153 Uc                 IDENT            Uc
154 )                  )
155 ;
156 ===== FOUND: PROCEDURE_CALL 'Put'=====
157 Uc                 IDENT            Uc
158 :=                 ASSIGN
159 Character           IDENT            Character
160 '
161 Succ               IDENT            Succ
162 (                  (
163 Uc                 IDENT            Uc
164 )                  )
165 ;
166 ===== FOUND: ASSIGNMENT 'Uc'=====
167 New_Line           IDENT            New_Line
168 ;
169 ===== FOUND: PROCEDURE_CALL 'New_Line'=====
170 end                KW.END
171 loop               KWLOOP
172 ;
173 ===== FOUND: FOR_LOOP =====
174 F1                 IDENT            F1
175 :=                 ASSIGN
176 1.1                FLOAT.CONST      1.1
177 *                  *
178 0.1                FLOAT.CONST      0.1
179 +                  +
180 1.0e-2             FLOAT.CONST      1.0e-2
181 ;
182 ===== FOUND: ASSIGNMENT 'F1'=====
183 T                  IDENT            T
184 (                  (
185 0                  INTEGER.CONST    0
186 )                  )
187 :=                 ASSIGN
188 0                  INTEGER.CONST    0
189 ;
190 ===== FOUND: ASSIGNMENT 'T'=====

```

```

191 for                KW.FOR
192 I                  IDENT          I
193 in                 KW.IN
194 1                  INTEGER.CONST  1
195 ..                 RANGE
196 10                 INTEGER.CONST  10
197 loop               KW.LOOP
198 T                  IDENT          T
199 (                  (
200 I                  IDENT          I
201 )                  )
202 :=                 ASSIGN
203 T                  IDENT          T
204 (                  (
205 I                  IDENT          I
206 )                  )
207 *                  *
208 I                  IDENT          I
209 *                  *
210 I                  IDENT          I
211 ;                  ;
212 ===== FOUND: ASSIGNMENT 'T'=====
213 end                 KW.END
214 loop               KW.LOOP
215 ;                  ;
216 ===== FOUND: FOR.LOOP =====
217 D                  IDENT          D
218 .                  .
219 Dzien              IDENT          Dzien
220 :=                 ASSIGN
221 1                  INTEGER.CONST  1
222 ;                  ;
223 ===== FOUND: ASSIGNMENT 'D.Dzien'=====
224 D                  IDENT          D
225 .                  .
226 Miesiac            IDENT          Miesiac
227 :=                 ASSIGN
228 D                  IDENT          D
229 .                  .
230 Dzien              IDENT          Dzien
231 *                  *
232 10                 INTEGER.CONST  10
233 ;                  ;
234 ===== FOUND: ASSIGNMENT 'D.Miesiac'=====
235 D                  IDENT          D
236 .                  .
237 Rok                IDENT          Rok
238 :=                 ASSIGN
239 2018               INTEGER.CONST  2018
240 ;                  ;
241 ===== FOUND: ASSIGNMENT 'D.Rok'=====
242 end                 KW.END
243 ListAscii          IDENT          ListAscii
244 ===== FOUND: BLOCK =====
245 ;                  ;
246 ===== FOUND: PROC.DEFINITION 'ListAscii'=====
247 end                 KW.END
248 MyTest             IDENT          MyTest
249 ;                  ;
250 ===== FOUND: PACKAGE.BODY 'MyTest'=====
251 use                 KW.USE
252 MyTest             IDENT          MyTest
253 ;                  ;
254 ===== FOUND: USE.CLAUSE =====
255 FromASCII          IDENT          FromASCII
256 :                  :

```

```

257 constant          KW.CONSTANT
258 Integer           IDENT          Integer
259 :=                ASSIGN
260 32                 INTEGER.CONST  32
261 ;
262 ===== FOUND: CONST.DECL 'FromASCII'=====
263 ToASCII           IDENT          ToASCII
264 :
265 constant          KW.CONSTANT
266 Integer           IDENT          Integer
267 :=                ASSIGN
268 127                INTEGER.CONST  127
269 ;
270 ===== FOUND: CONST.DECL 'ToASCII'=====
271 begin             KW.BEGIN
272 if                KW.IF
273 FromAscii         IDENT          FromAscii
274 <=                LE
275 ToAscii           IDENT          ToAscii
276 then              KW.THEN
277 ListAscii         IDENT          ListAscii
278 (                 (
279 FromAscii         IDENT          FromAscii
280 ,
281 ToAscii           IDENT          ToAscii
282 )                 )
283 ;
284 ===== FOUND: PROCEDURE.CALL 'ListAscii'=====
285 else              KW.ELSE
286 Put_Line          IDENT          Put_Line
287 (                 (
288 "Figa"            STRING.CONST   "Figa"
289 )                 )
290 ;
291 ===== FOUND: PROCEDURE.CALL 'Put_Line'=====
292 New_Line          IDENT          New_Line
293 ;
294 ===== FOUND: PROCEDURE.CALL 'New_Line'=====
295 end               KW.END
296 if                KW.IF
297 ;
298 ===== FOUND: IF.STATEMENT =====
299 end               KW.END
300 Test              IDENT          Test
301 ===== FOUND: BLOCK =====
302 ;
303 ===== FOUND: PROC.DEFINITION 'Test'=====

```